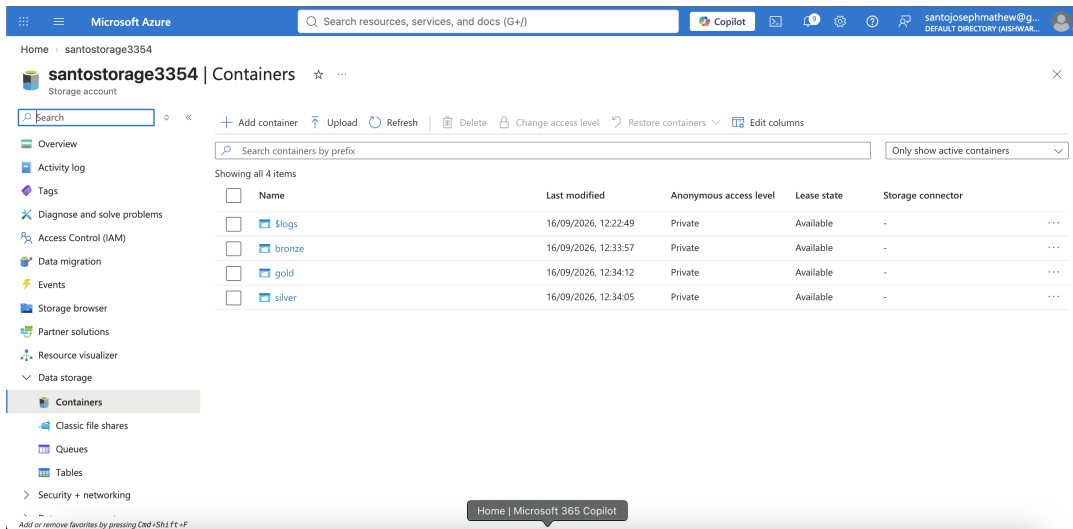


Azure + Databricks Medallion Pipeline

Step-by-step process: Bronze → Silver → Gold data pipeline using Azure Storage, Microsoft Entra service principal auth, and Azure Databricks / Spark.

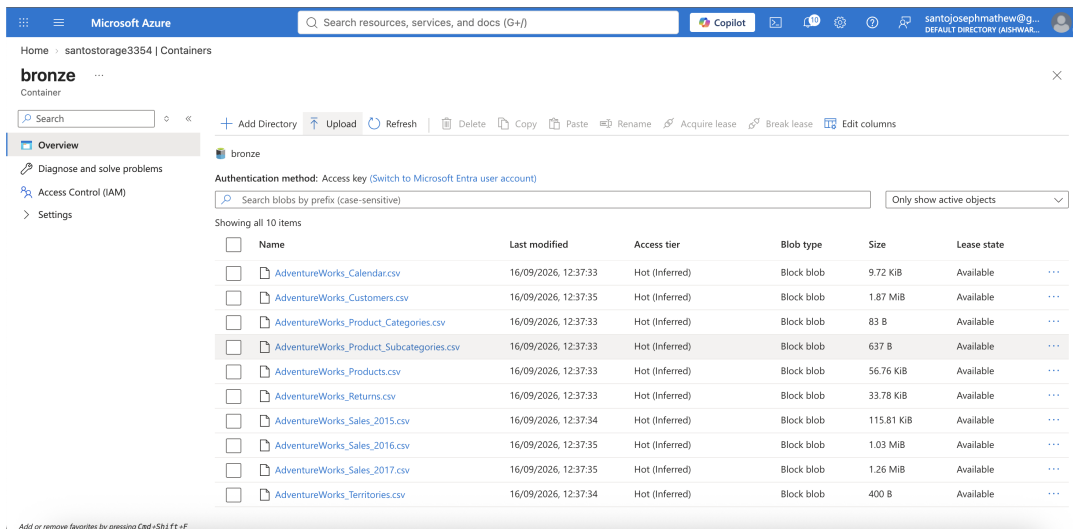
1. Set up Azure Storage with medallion containers

Created a storage account (storagesanto / santostorage3354) with four containers: \$logs, bronze, silver, and gold — the classic bronze/silver/gold medallion layout for a data lake.



2. Load raw CSVs into the bronze container

Uploaded the raw AdventureWorks CSV files (Calendar, Customers, Products, Sales by year, Returns, Territories, etc.) directly into the bronze container as-is, untransformed.



3. Register an app in Microsoft Entra ID

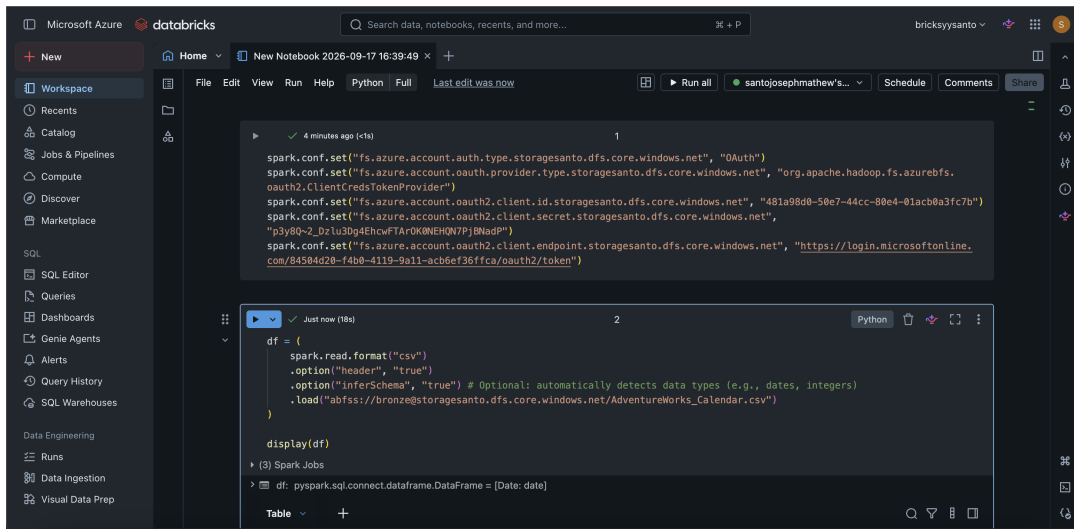
Created an app registration ('Appregistration') to act as a service principal, capturing its Application (client) ID and Directory (tenant) ID, and generating a client secret under Certificates & secrets.

The top screenshot shows the 'Default Directory | Overview' page in the Microsoft Azure portal. The page displays basic information about the directory, including its name, tenant ID, primary domain, and license. The 'My feed' section shows a notification about the new Microsoft Entra admin center.

The bottom screenshot shows the 'App registration' page in the Microsoft Azure portal. The page displays details for a newly created application, including its display name, client ID, object ID, directory (tenant) ID, and supported account types. The 'Get Started' section provides a link to the documentation.

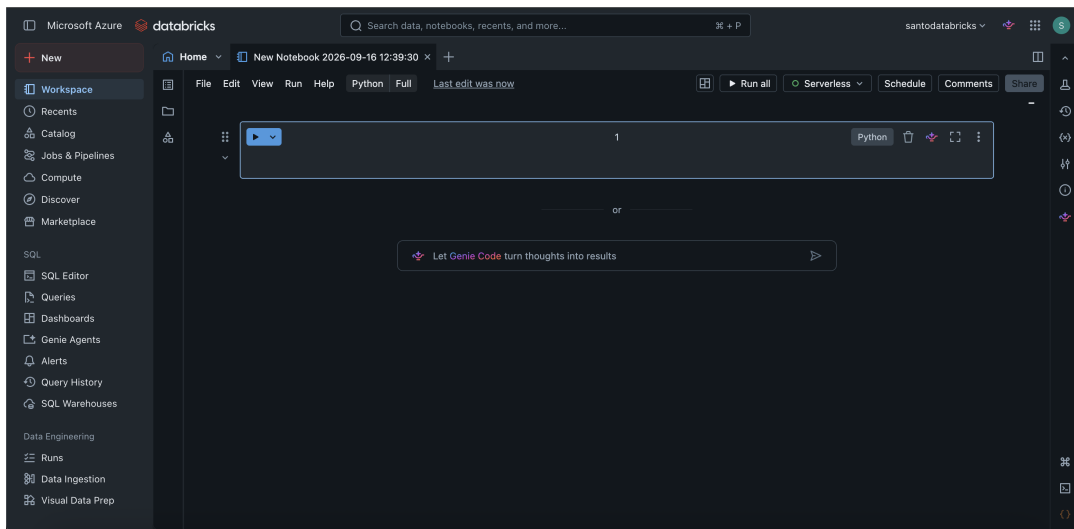
4. Grant the service principal storage access

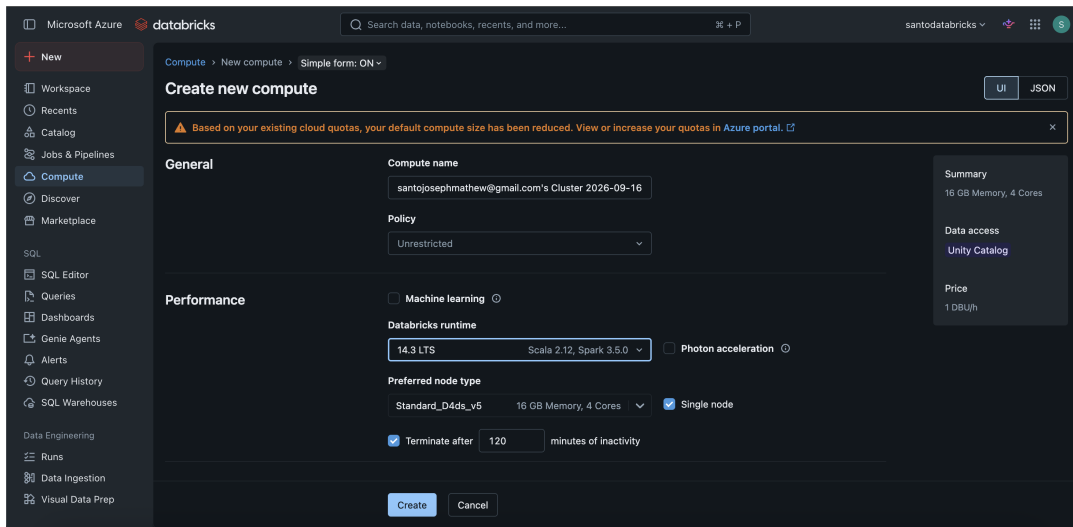
In the storage account's (or resource group's) Access Control (IAM), assigned the 'Storage Blob Data Contributor' role to the app/service principal (AzureDatabricks), so it can read and write blobs programmatically.



5. Spin up an Azure Databricks workspace and cluster

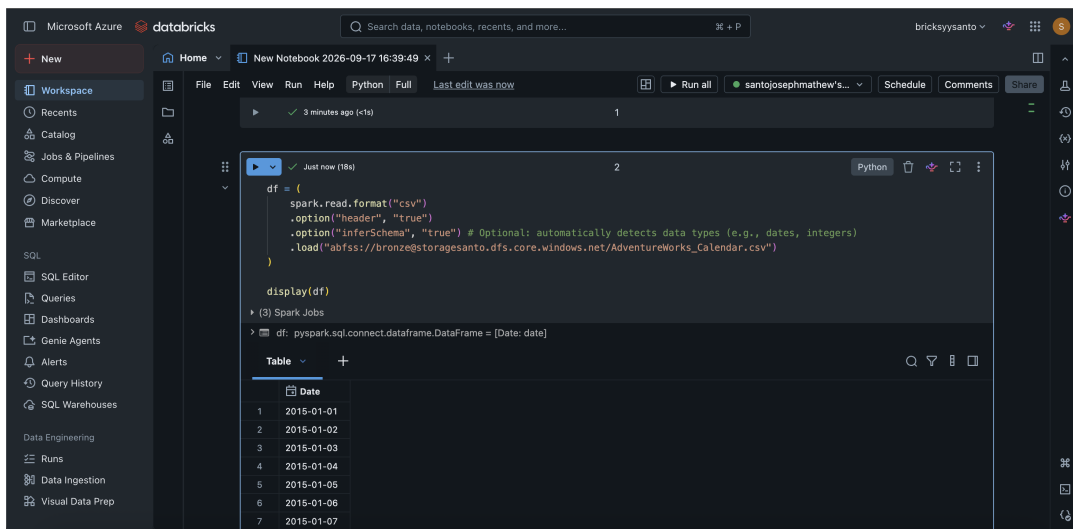
Created a Databricks workspace and a single-node compute cluster (Standard_D4ds_v5, Databricks Runtime 14.3 LTS) to run notebooks.





6. Authenticate Spark to ADLS via OAuth

In a notebook, configured `spark.conf.set()` calls to enable OAuth auth on the storage account using the service principal's client ID, secret, and tenant token endpoint — giving Spark direct access to `abfss://` paths.



7. Read bronze data and transform to silver

Read the bronze CSVs into Spark DataFrames (`spark.read.format('csv').load(...)`), then cleaned/transformed each dataset and wrote it out as its own folder in the silver container (Calendar, Customers, Products, Sales, Sales_Grouped, etc.).

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

santojosephmathew@g...
DEFAULT DIRECTORY (ADPWAR...)

Home > Resource Manager | Resource groups > Akunem | Access control (IAM)

Add role assignment

Role: **Members** Conditions Review + assign

Selected role Storage Blob Data Contributor

Assign access to ☒ User, group, or service principal ☐ Managed identity

Members + Select members

Name	Object ID	Type
AzureDataBricks	af6dca95-6fd4-40d7-9a54-7ba3ec5f4c2c	App

Description Optional

Review + assign Previous Next

Feedback

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

santojosephmathew@g...
DEFAULT DIRECTORY (ADPWAR...)

Home > storagesanto | Containers

silver

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview Diagnose and solve problems Access Control (IAM) Settings

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive) Only show active objects

Showing all 9 items

Name	Last modified	Access tier	Blob type	Size	Lease state
Calendar	20/09/2026, 22:39:00				...
Customers	20/09/2026, 22:35:19				...
Product_Categories	20/09/2026, 22:39:14				...
Product_Subcategories	20/09/2026, 22:35:38				...
Products	20/09/2026, 22:35:54				...
Returns	20/09/2026, 22:39:23				...
Sales	20/09/2026, 22:36:07				...
Sales_Grouped	20/09/2026, 22:36:20				...
Territories	20/09/2026, 22:39:31				...

Add or remove favorites by pressing Cmd+Shift+F or F6

8. Aggregate and publish to gold

Built curated, business-ready aggregates (e.g., Sales_Gold) from the silver tables and wrote them out as Delta format in the gold container, producing part files plus a `_delta_log` for versioned, query-ready data.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

santosephmathew@g...
DEFAULT DIRECTORY (ADPWAR...

Home > storagesanto | Containers

gold
Container

Search

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

gold

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 1 items

Name	Last modified	Access tier	Blob type	Size	Lease state
Sales_Gold	20/09/2026, 22:57:29				

Add or remove favorites by pressing Cmd+Shift+F.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

santosephmathew@g...
DEFAULT DIRECTORY (ADPWAR...

Home > storagesanto | Containers

gold
Container

Search

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

gold > Sales_Gold

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 4 items

Name	Last modified	Access tier	Blob type	Size	Lease state
[.]					
_delta_log	20/09/2026, 22:57:29				
part-00000-0acfa6c0-a63d-4eb2-adf0-70245ec7e0...	20/09/2026, 22:57:31	Hot (Inferred)	Block blob	97.4 KiB	Available
part-00001-f8d0f13c-924c-4a12-9ad8-72f298260d...	20/09/2026, 22:57:32	Hot (Inferred)	Block blob	754.4 KiB	Available
part-00002-2dbeb50c-ce6d-4740-9a5b-a3dad466...	20/09/2026, 22:57:32	Hot (Inferred)	Block blob	893.55 KiB	Available

Add or remove favorites by pressing Cmd+Shift+F.

Appendix: Full PySpark Notebook Code

AdventureWorks_Data_Quality.ipynb — all code cells, in order.

Cell 1

```
spark.conf.set("fs.azure.account.auth.type.storagesanto.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.storagesanto.dfs.core.windows.net",
               "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id.storagesanto.dfs.core.windows.net",
               "481a98d0-50e7-44cc-80e4-01acb0a3fc7b")
spark.conf.set("fs.azure.account.oauth2.client.secret.storagesanto.dfs.core.windows.net",
               "p3y8Q~2_Dzlu3Dg4EhcwFTArOK0K0NEHQN7PjBNadP")
spark.conf.set("fs.azure.account.oauth2.client.endpoint.storagesanto.dfs.core.windows.net",
               "https://login.microsoftonline.com/84504d20-f4b0-4119-9a11-acb6ef36ffca/oauth2/token")
```

Cell 2

```
df = (
    spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true") # Optional: automatically detects data types (e.g., dates,
        integers)
        .load("abfss://bronze@storagesanto.dfs.core.windows.net/AdventureWorks_Calendar.csv")
)

display(df)
```

Cell 3

```
display(
    dbutils.fs.ls(
        "abfss://bronze@storagesanto.dfs.core.windows.net/"
    )
)
```

Cell 4

```
files = dbutils.fs.ls(
    "abfss://bronze@storagesanto.dfs.core.windows.net/"
)

for file in files:
    print(file.name)
```

Cell 5

```
bronze_path = "abfss://bronze@storagesanto.dfs.core.windows.net/"

calendar_df = (
    spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(bronze_path + "AdventureWorks_Calendar.csv")
)

customers_df = (
    spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(bronze_path + "AdventureWorks_Customers.csv")
)

product_categories_df = (
    spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(bronze_path + "AdventureWorks_Product_Categories.csv")
)

product_subcategories_df = (
```

```

        spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(bronze_path + "AdventureWorks_Product_Subcategories.csv")
    )

products_df = (
    spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load(bronze_path + "AdventureWorks_Products.csv")
)

returns_df = (
    spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load(bronze_path + "AdventureWorks_Returns.csv")
)

sales_2015_df = (
    spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load(bronze_path + "AdventureWorks_Sales_2015.csv")
)

sales_2016_df = (
    spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load(bronze_path + "AdventureWorks_Sales_2016.csv")
)

sales_2017_df = (
    spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load(bronze_path + "AdventureWorks_Sales_2017.csv")
)

territories_df = (
    spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load(bronze_path + "AdventureWorks_Territories.csv")
)

```

Cell 6

```

print("Calendar:", calendar_df.count())
print("Customers:", customers_df.count())
print("Product Categories:", product_categories_df.count())
print("Product Subcategories:", product_subcategories_df.count())
print("Products:", products_df.count())
print("Returns:", returns_df.count())
print("Sales 2015:", sales_2015_df.count())
print("Sales 2016:", sales_2016_df.count())
print("Sales 2017:", sales_2017_df.count())
print("Territories:", territories_df.count())

```

Cell 7

```

print("==== CALENDAR ====")
display(calendar_df)

print("==== CUSTOMERS ====")
display(customers_df)

print("==== PRODUCT CATEGORIES ====")
display(product_categories_df)

```

```

print("===== PRODUCT SUBCATEGORIES =====")
display(product_subcategories_df)

print("===== PRODUCTS =====")
display(products_df)

print("===== RETURNS =====")
display(returns_df)

print("===== SALES 2015 =====")
display(sales_2015_df)

print("===== SALES 2016 =====")
display(sales_2016_df)

print("===== SALES 2017 =====")
display(sales_2017_df)

print("===== TERRITORIES =====")
display(territories_df)

```

Cell 8

```

import builtins
from pyspark.sql import Row
from pyspark.sql.functions import col, sum as spark_sum

tables = {
    "Calendar": calendar_df,
    "Customers": customers_df,
    "Product Categories": product_categories_df,
    "Product Subcategories": product_subcategories_df,
    "Products": products_df,
    "Returns": returns_df,
    "Sales 2015": sales_2015_df,
    "Sales 2016": sales_2016_df,
    "Sales 2017": sales_2017_df,
    "Territories": territories_df
}

quality_report = []

for name, df in tables.items():

    total_rows = df.count()
    total_columns = len(df.columns)

    distinct_rows = df.dropDuplicates().count()
    duplicate_rows = total_rows - distinct_rows

    null_row = df.select([
        spark_sum(col(c).isNull().cast("int")).alias(c)
        for c in df.columns
    ]).collect()[0]

    total_nulls = builtins.sum(
        (null_row[c] or 0)
        for c in df.columns
    )

    quality_report.append(
        Row(
            Table=name,
            Rows=total_rows,
            Columns=total_columns,
            Duplicate_Rows=duplicate_rows,
            Total_Nulls=total_nulls
        )
    )

quality_df = spark.createDataFrame(quality_report)

```

```
display(quality_df)
```

Cell 9

```
# Create Silver DataFrames from Bronze

calendar_silver = calendar_df.dropDuplicates()

customers_silver = customers_df.dropDuplicates()

product_categories_silver = product_categories_df.dropDuplicates()

product_subcategories_silver = product_subcategories_df.dropDuplicates()

products_silver = products_df.dropDuplicates()

returns_silver = returns_df.dropDuplicates()

sales_2015_silver = sales_2015_df.dropDuplicates()

sales_2016_silver = sales_2016_df.dropDuplicates()

sales_2017_silver = sales_2017_df.dropDuplicates()

territories_silver = territories_df.dropDuplicates()

print("Silver DataFrames created successfully.")
```

Cell 10

```
sales_silver = (
    sales_2015_silver
    .unionByName(sales_2016_silver)
    .unionByName(sales_2017_silver)
)

print("Total Sales rows:", sales_silver.count())

display(sales_silver)
```

Cell 11

```
print("CUSTOMERS")
print(customers_silver.columns)

print("\nPRODUCTS")
print(products_silver.columns)

print("\nSALES")
print(sales_silver.columns)
```

Cell 12

```
from pyspark.sql.functions import concat_ws, trim, col

# Create fresh Customers Silver DataFrame
customers_silver = customers_df.dropDuplicates()

# Create FullName using the ACTUAL column names
customers_silver = customers_silver.withColumn(
    "FullName",
    trim(
        concat_ws(
            " ",
            col("Prefix"),
            col("FirstName"),
            col("LastName")
        )
    )
)

display(customers_silver)
```

Cell 13

```
customers_silver.select(
    "Prefix",
    "FirstName",
    "LastName",
    "FullName"
).show(10, False)
```

Cell 14

```
print(products_silver.columns)
```

Cell 15

```
from pyspark.sql.functions import split, col

products_silver = products_silver.withColumn(
    "ProductSKU_Split",
    split(col("ProductSKU"), "-")[0]
)

display(products_silver)
```

Cell 16

```
print(products_silver.columns)
```

Cell 17

```
from pyspark.sql.functions import split, col

products_silver = products_silver.withColumn(
    "ProductName_Split",
    split(col("ProductName"), "-")[0]
)

display(
    products_silver.select(
        "ProductName",
        "ProductName_Split"
    )
)
```

Cell 18

```
products_silver.printSchema()
```

Cell 19

```
print(sales_silver.columns)
```

Cell 20

```
from pyspark.sql.functions import col, regexp_replace

sales_silver = sales_silver.withColumn(
    "OrderNumber",
    regexp_replace(
        col("OrderNumber"),
        "^S",
        "D"
    )
)

display(sales_silver.select("OrderNumber"))
```

Cell 21

```
from pyspark.sql.functions import to_timestamp

sales_silver = sales_silver.withColumn(
    "StockDate",
    to_timestamp(col("StockDate"))
)
```

```
sales_silver.printSchema()
```

Cell 22

```
from pyspark.sql.functions import sum, count

sales_grouped = (
    sales_silver
    .groupBy(
        "OrderDate",
        "OrderNumber"
    )
    .agg(
        sum("OrderQuantity").alias("TotalOrderQuantity"),
        count("*").alias("TotalOrderLines")
    )
)

display(sales_grouped)
```

Cell 23

```
print("Customers:", customers_silver.count())
print("Products:", products_silver.count())
print("Product Subcategories:", product_subcategories_silver.count())
print("Sales:", sales_silver.count())
print("Sales Grouped:", sales_grouped.count())
```

Cell 24

```
silver_path = "abfss://silver@storagesanto.dfs.core.windows.net/"
```

Cell 25

```
customers_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save(silver_path + "Customers")
```

Cell 26

```
product_subcategories_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save(silver_path + "Product_Subcategories")
```

Cell 27

```
products_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save(silver_path + "Products")
```

Cell 28

```
sales_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save(silver_path + "Sales")
```

Cell 29

```
sales_grouped.write \
    .format("delta") \
    .mode("overwrite") \
    .save(silver_path + "Sales_Grouped")
```

Cell 30

```
display(
    spark.read.format("delta")
    .load("abfss://silver@storagesanto.dfs.core.windows.net/Customers")
)
```

Cell 31

```
display(
    spark.read.format("delta")
        .load("abfss://silver@storagesanto.dfs.core.windows.net/Products")
)
```

Cell 32

```
display(
    spark.read.format("delta")
        .load("abfss://silver@storagesanto.dfs.core.windows.net/Product_Subcategories")
)
```

Cell 33

```
display(
    spark.read.format("delta")
        .load("abfss://silver@storagesanto.dfs.core.windows.net/Sales")
)
```

Cell 34

```
display(
    spark.read.format("delta")
        .load("abfss://silver@storagesanto.dfs.core.windows.net/Sales_Grouped")
)
```

Cell 35

```
calendar_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save("abfss://silver@storagesanto.dfs.core.windows.net/Calendar")
```

Cell 36

```
product_categories_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save("abfss://silver@storagesanto.dfs.core.windows.net/Product_Categories")
```

Cell 37

```
returns_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save("abfss://silver@storagesanto.dfs.core.windows.net>Returns")
```

Cell 38

```
territories_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .save("abfss://silver@storagesanto.dfs.core.windows.net/Territories")
```

Cell 39

```
silver_paths = {
    "Calendar": "Calendar",
    "Customers": "Customers",
    "Product Categories": "Product_Categories",
    "Product Subcategories": "Product_Subcategories",
    "Products": "Products",
    "Returns": "Returns",
    "Sales": "Sales",
    "Sales Grouped": "Sales_Grouped",
    "Territories": "Territories"
}

for name, folder in silver_paths.items():
    df = spark.read.format("delta").load(
        "abfss://silver@storagesanto.dfs.core.windows.net/" + folder
    )
```

```

print(
    f"{name}: {df.count()} rows, "
    f"{len(df.columns)} columns"
)

```

Cell 40

```

from pyspark.sql.functions import col, sum

silver_tables = {
    "Calendar": calendar_silver,
    "Customers": customers_silver,
    "Product Categories": product_categories_silver,
    "Product Subcategories": product_subcategories_silver,
    "Products": products_silver,
    "Returns": returns_silver,
    "Sales": sales_silver,
    "Sales Grouped": sales_grouped,
    "Territories": territories_silver
}

for name, df in silver_tables.items():
    print("=" * 60)
    print(name)
    print("Rows:", df.count())
    print("Columns:", len(df.columns))

    nulls = df.select([
        sum(col(c).isNull().cast("int")).alias(c)
        for c in df.columns
    ])

    display(nulls)

```

Cell 41

```

display(
    customers_silver.select(
        "CustomerKey",
        "Prefix",
        "FirstName",
        "LastName",
        "FullName"
    )
)

```

Cell 42

```

display(
    products_silver.select(
        "ProductSKU",
        "ProductSKU_Split",
        "ProductName",
        "ProductName_Split"
    )
)

```

Cell 43

```

display(
    sales_silver.select(
        "OrderDate",
        "StockDate",
        "OrderNumber",
        "OrderQuantity"
    )
)

```

Cell 44

```

silver_path = "abfss://silver@storagesanto.dfs.core.windows.net/"

calendar = spark.read.format("delta").load(silver_path + "Calendar")

```

```

customers = spark.read.format("delta").load(silver_path + "Customers")
categories = spark.read.format("delta").load(silver_path + "Product_Categories")
subcategories = spark.read.format("delta").load(silver_path + "Product_Subcategories")
products = spark.read.format("delta").load(silver_path + "Products")
returns = spark.read.format("delta").load(silver_path + "Returns")
sales = spark.read.format("delta").load(silver_path + "Sales")
territories = spark.read.format("delta").load(silver_path + "Territories")

print("All Silver tables loaded successfully.")

```

Cell 45

```

print("SALES")
print(sales.columns)

print("\nCUSTOMERS")
print(customers.columns)

print("\nPRODUCTS")
print(products.columns)

print("\nSUBCATEGORIES")
print(subcategories.columns)

print("\nCATEGORIES")
print(categories.columns)

print("\nTERRITORIES")
print(territories.columns)

print("\nCALENDAR")
print(calendar.columns)

print("\nRETURNS")
print(returns.columns)

```

Cell 46

```

from pyspark.sql.functions import col

gold_sales = (
    sales.alias("s")
    .join(
        customers.alias("c"),
        col("s.CustomerKey") == col("c.CustomerKey"),
        "left"
    )
    .join(
        products.alias("p"),
        col("s.ProductKey") == col("p.ProductKey"),
        "left"
    )
    .join(
        subcategories.alias("sc"),
        col("p.ProductSubcategoryKey") == col("sc.ProductSubcategoryKey"),
        "left"
    )
    .join(
        categories.alias("cat"),
        col("sc.ProductCategoryKey") == col("cat.ProductCategoryKey"),
        "left"
    )
)

display(gold_sales)

```

Cell 47

```

gold_sales = gold_sales.select(
    col("s.OrderDate"),
    col("s.StockDate"),
    col("s.OrderNumber"),

```

```

    col("s.OrderLineItem"),
    col("s.OrderQuantity"),

    col("c.CustomerKey"),
    col("c.FullName"),
    col("c.Gender"),
    col("c.AnualIncome"),
    col("c.Occupation"),

    col("p.ProductKey"),
    col("p.ProductSKU"),
    col("p.ProductSKU_Split"),
    col("p.ProductName"),
    col("p.ProductName_Split"),
    col("p.ModelName"),
    col("p.ProductColor"),
    col("p.ProductSize"),
    col("p.ProductCost"),
    col("p.ProductPrice"),

    col("sc.ProductSubcategoryKey"),
    col("sc.SubcategoryName"),

    col("cat.ProductCategoryKey"),
    col("cat.CategoryName")
)

display(gold_sales)

```

Cell 48

```

from pyspark.sql.functions import round

gold_sales = gold_sales.withColumn(
    "SalesAmount",
    round(col("OrderQuantity") * col("ProductPrice"), 2)
)

display(gold_sales)

```

Cell 49

```

gold_sales = gold_sales.withColumn(
    "TotalCost",
    round(col("OrderQuantity") * col("ProductCost"), 2)
)

```

Cell 50

```

gold_sales = gold_sales.withColumn(
    "GrossProfit",
    round(col("SalesAmount") - col("TotalCost"), 2)
)

display(gold_sales)

```

Cell 51

```

print("TERRITORIES:")
print(territories.columns)

print("\nCALENDAR:")
print(calendar.columns)

print("\nRETURNS:")
print(returns.columns)

```

Cell 52

```

from pyspark.sql.functions import col, round

gold_sales = (
    sales.alias("s")

```

```

.join(
  customers.alias("c"),
  col("s.CustomerKey") == col("c.CustomerKey"),
  "left"
)
.join(
  products.alias("p"),
  col("s.ProductKey") == col("p.ProductKey"),
  "left"
)
.join(
  subcategories.alias("sc"),
  col("p.ProductSubcategoryKey") == col("sc.ProductSubcategoryKey"),
  "left"
)
.join(
  categories.alias("cat"),
  col("sc.ProductCategoryKey") == col("cat.ProductCategoryKey"),
  "left"
)
)
.select(
  col("s.OrderDate"),
  col("s.StockDate"),
  col("s.OrderNumber"),
  col("s.OrderLineItem"),
  col("s.OrderQuantity"),
  col("s.TerritoryKey"),

  col("c.CustomerKey"),
  col("c.FullName"),
  col("c.Gender"),
  col("c.AnualIncome"),
  col("c.Occupation"),

  col("p.ProductKey"),
  col("p.ProductSKU"),
  col("p.ProductSKU_Split"),
  col("p.ProductName"),
  col("p.ProductName_Split"),
  col("p.ModelName"),
  col("p.ProductColor"),
  col("p.ProductSize"),
  col("p.ProductCost"),
  col("p.ProductPrice"),

  col("sc.ProductSubcategoryKey"),
  col("sc.SubcategoryName"),

  col("cat.ProductCategoryKey"),
  col("cat.CategoryName")
)
)

# Calculate business metrics
gold_sales = (
  gold_sales
  .withColumn(
    "SalesAmount",
    round(col("OrderQuantity") * col("ProductPrice"), 2)
  )
  .withColumn(
    "TotalCost",
    round(col("OrderQuantity") * col("ProductCost"), 2)
  )
  .withColumn(
    "GrossProfit",
    round(col("SalesAmount") - col("TotalCost"), 2)
  )
)
)

```

```
display(gold_sales)
```

Cell 53

```
print(gold_sales.columns)
```

Cell 54

```
from pyspark.sql.functions import col

gold_sales_final = (
    gold_sales.alias("g")
    .join(
        territories.alias("t"),
        col("g.TerritoryKey") == col("t.SalesTerritoryKey"),
        "left"
    )
)

display(gold_sales_final)
```

Cell 55

```
gold_sales_final = (
    gold_sales_final.alias("g")
    .join(
        calendar.alias("cal"),
        col("g.OrderDate") == col("cal.Date"),
        "left"
    )
)

display(gold_sales_final)
```

Cell 56

```
gold_sales_final = (
    gold_sales_final.alias("g")
    .join(
        returns.alias("r"),
        (col("g.ProductKey") == col("r.ProductKey")) &
        (col("g.TerritoryKey") == col("r.TerritoryKey")) &
        (col("g.OrderDate") == col("r.ReturnDate")),
        "left"
    )
    .select(
        col("g.*"),
        col("r.ReturnQuantity")
    )
)

display(gold_sales_final)
```

Cell 57

```
from pyspark.sql.functions import coalesce, lit, col

gold_sales_final = gold_sales_final.withColumn(
    "ReturnQuantity",
    coalesce(col("ReturnQuantity"), lit(0))
)

display(gold_sales_final)
```

Cell 58

```
print(gold_sales_final.columns)
```

Cell 59

```
gold_path = "abfss://gold@storagesanto.dfs.core.windows.net/"

gold_sales_final.write \
    .format("delta") \
```

```
.mode("overwrite") \  
.save(gold_path + "Sales_Gold")
```

Cell 60

```
gold_sales_check = (  
    spark.read  
    .format("delta")  
    .load(gold_path + "Sales_Gold")  
)  
  
display(gold_sales_check)
```

Cell 61

```
print("Gold row count:", gold_sales_check.count())
```

Cell 62

```
from pyspark.sql.functions import col, sum, count, countDistinct, avg, max, min  
  
print("Total Rows:", gold_sales_check.count())  
  
print("Total Orders:", gold_sales_check.select(  
    countDistinct("OrderNumber")  
)  
.collect()[0][0])  
  
print("Total Customers:", gold_sales_check.select(  
    countDistinct("CustomerKey")  
)  
.collect()[0][0])  
  
print("Total Products:", gold_sales_check.select(  
    countDistinct("ProductKey")  
)  
.collect()[0][0])
```

Cell 63

```
gold_kpis = gold_sales_check.select(  
    sum("SalesAmount").alias("Total Sales"),  
    sum("TotalCost").alias("Total Cost"),  
    sum("GrossProfit").alias("Total Gross Profit"),  
    sum("OrderQuantity").alias("Total Quantity Sold"),  
    sum("ReturnQuantity").alias("Total Quantity Returned"),  
    countDistinct("OrderNumber").alias("Total Orders"),  
    countDistinct("CustomerKey").alias("Total Customers")  
)  
  
display(gold_kpis)
```

Cell 64

```
sales_by_category = (  
    gold_sales_check  
    .groupBy("CategoryName")  
    .agg(  
        sum("SalesAmount").alias("Total Sales"),  
        sum("GrossProfit").alias("Total Profit"),  
        sum("OrderQuantity").alias("Quantity Sold")  
    )  
    .orderBy(col("Total Sales").desc())  
)  
  
display(sales_by_category)
```

Cell 65

```
sales_by_region = (  
    gold_sales_check  
    .groupBy("Region")  
    .agg(  
        sum("SalesAmount").alias("Total Sales"),  
        sum("GrossProfit").alias("Total Profit"),  
        sum("OrderQuantity").alias("Quantity Sold")  
    )  
)
```

```

    )
    .orderBy(col("Total Sales").desc())
)

display(sales_by_region)

```

Cell 66

```

from pyspark.sql.functions import sum, countDistinct

gold_kpis = gold_sales_check.select(
    sum("SalesAmount").alias("Total Sales"),
    sum("TotalCost").alias("Total Cost"),
    sum("GrossProfit").alias("Total Gross Profit"),
    sum("OrderQuantity").alias("Total Quantity Sold"),
    sum("ReturnQuantity").alias("Total Quantity Returned"),
    countDistinct("OrderNumber").alias("Total Orders"),
    countDistinct("CustomerKey").alias("Total Customers")
)

display(gold_kpis)

```

Cell 67

```

from pyspark.sql.functions import sum, col

sales_by_category = (
    gold_sales_check
    .groupBy("CategoryName")
    .agg(
        sum("SalesAmount").alias("Total Sales"),
        sum("GrossProfit").alias("Total Profit"),
        sum("OrderQuantity").alias("Quantity Sold")
    )
    .orderBy(col("Total Sales").desc())
)

display(sales_by_category)

```

Cell 68

```

from pyspark.sql.functions import sum, col

sales_by_region = (
    gold_sales_check
    .groupBy("Region")
    .agg(
        sum("SalesAmount").alias("Total Sales"),
        sum("GrossProfit").alias("Total Profit"),
        sum("OrderQuantity").alias("Quantity Sold")
    )
    .orderBy(col("Total Sales").desc())
)

display(sales_by_region)

```